

Lab 3: Altitude Estimation with a Kalman Filter

Abstract

Relying on a single sensor for UAV state estimation is insufficient. Barometers provide an absolute altitude but suffer from noise, drift, and aerodynamic disturbances. Accelerometers react instantly to motion but suffer from unbounded drift when integrated over time. In this lab, a Kalman Filter to fuse barometer static pressure measurements with specific force data from the IMU, resulting in a smooth and responsive altitude estimate.

Key Objectives:

- **Task 1:** Log high-speed asynchronous data from the UAV IMU and Barometer.
- **Task 2:** Perform coordinate frame transformations (Body to Earth frame) to isolate the vertical acceleration component.
- **Task 3:** Implement a discrete-time Kalman Filter (Predict & Update loop) in Python to estimate the altitude and vertical velocity.

Sensor Fusion & System Model

Important: This lab is building on the Python environment from the previous lab to capture data.

The Kalman Filter estimates the state $\vec{x}_k$ based on a linear model. The state vector contains the altitude/position (z), vertical velocity (v_z), and an accelerometer bias on the vertical axis (b_a as nuisance variable):

$$\vec{x} = \begin{bmatrix} z \\ v_z \\ b_a \end{bmatrix}$$

1. The Prediction Step (Kinematic Model):

We use the vertical acceleration in the navigation frame a_z^n as the control input u_k . The prediction steps are governed by the physical equations of motion:

$$\begin{aligned} \vec{x}_k^- &= \mathbf{A}_{k-1} \cdot \vec{x}_{k-1}^+ + \mathbf{B}u_{k-1} \\ \vec{x}_k^- &= \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} z_{k-1} \\ v_{z,k-1} \\ b_{a,k-1} \end{bmatrix} + \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 0 \end{bmatrix} (a_{z,k-1}^n + g^n) \end{aligned}$$

The covariance propagation introduces the process noise matrix $\mathbf{Q}$:

$$\mathbf{P}_k^- = \mathbf{A}\mathbf{P}_{k-1}^+ \mathbf{A}^\top + \mathbf{Q}$$

2. The Update Step (Barometer Measurement):

The measurement y_k is the barometer reading, which directly observes the state z via the observation matrix $\mathbf{H}$:

$$y_k = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \cdot \vec{x}_k$$

The Kalman Gain $\mathbf{K}_k$ weighs the trust between the prediction and the noisy measurement (using sensor noise variance R):

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + R_k)^{-1}$$
$$\vec{x}_k^+ = \vec{x}_k^- + \mathbf{K}_k (y_k - \mathbf{H}_k \vec{x}_k^-)$$

Preparation

- Use the previous lab Python environment to create a Python data logger script.
- Use the current attitude quaternion `stateEstimate.qw`, `stateEstimate.qx`, `stateEstimate.qy`, `stateEstimate.qz` to transform the accelerometer measurement from the body frame (b) to the navigation frame (n).
- Remember to correct the gravitational acceleration ($1\text{ G} \approx 9.80665\text{ m/s}^2$) after rotating the vector to isolate the true acceleration.
- **Hint:** Log the onboard estimated altitude (`stateEstimate.z`) to have a ground-truth baseline for your own filter.

Lab Tasks

1. **Data Acquisition:** Write a Python script using `cflib` to log the following variables to a CSV file at a high rate (e.g., 50 Hz or 100 Hz):

- `baro.asl` (Barometer altitude)
- `acc.x`, `acc.y`, `acc.z` (Body accelerations are in G not $\frac{m}{s^2}$)
- `stateEstimate.qw`, `.qx`, `.qy`, `.qz` (Attitude from body to navigation frame)
- `stateEstimate.z` (Onboard reference altitude for comparison)
- Link to parameters: [LINK](#)

Connect to the Crazyflie, start logging, and physically move the drone up and down by hand (varying speeds, pausing in between) for about 30 seconds.

2. **Offset correction:** Estimate the barometer offset from the first seconds of the captured data on the ground so that the estimated altitude is comparable to the reference lighthouse altitude (which is $\approx 0\text{ m}$ on ground).
3. **Coordinate Transformation:** In an offline Python analysis script (using `numpy` and `pandas`), read your CSV file. For every time step, calculate the true vertical Earth-frame acceleration a_z . *Be careful with units and the direction of the Z-axis (upwards vs. downwards).*
4. **Kalman Filter Implementation:** Implement the Predict/Update loops described in the Sensor Fusion & System Model section.
 - Initialize the state $\vec{x}_0$ with the first barometer reading and $v_z = 0$.
 - Choose reasonable initial values for the covariance matrix $\mathbf{P}$.
 - Tune the process noise covariance $\mathbf{Q}$ (trust in IMU/kinematics) and the measurement noise variance R (trust in the barometer).
5. **Evaluation & Plotting:** Use `matplotlib` to plot three graphs overlaid on the same time axis:

- (a) The raw, noisy barometer measurements (offset corrected to be aligned with `stateEstimate.z`).
- (b) The altitude estimate from your custom Kalman Filter, including a $1\text{-}\sigma$ error band.
- (c) The onboard `stateEstimate.z` reference (ground truth).

Deliverables

- **Code:** The data-logging script and the Kalman Filter analysis script.
- **Plot:** A clear, labeled plot showing the raw data and your smoothed estimate responding to rapid altitude changes.
- **Discussion:** A short written explanation covering:
 - How is the subtraction of gravity handled?
 - What values did you ultimately choose for $\mathbf{Q}$ and R , and how did changing them affect the "lag" vs. "smoothness" of your plot?

Background / Theory: Coordinate Transformation

The accelerometer measures specific force in the body frame (b). To use this in the navigation frame (n), the vector $\vec{a}^b = [a_x, a_y, a_z]^\top$ must be transformed. The attitude **Quaternion** $\mathbf{q} = [q_w, q_x, q_y, q_z]^\top$ is given by the state estimation of the UAV. To isolate the vertical Earth-frame accelerometer measurement a_z^n , apply the third row of the quaternion rotation matrix:

$$a_z^n = 2(q_x q_z - q_w q_y) a_x + 2(q_y q_z + q_w q_x) a_y + (q_w^2 - q_x^2 - q_y^2 + q_z^2) a_z$$

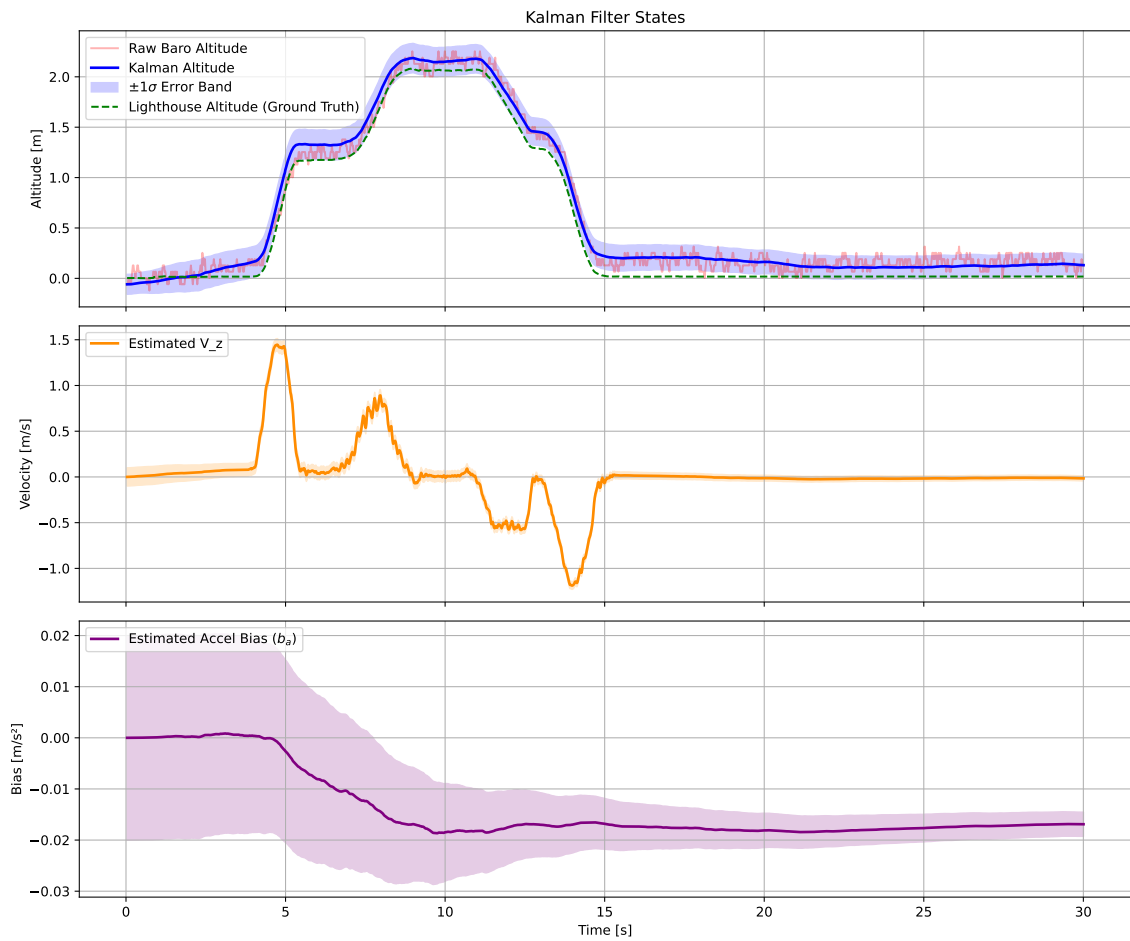


Figure 1: Example result: The Kalman Filter estimates a smooth altitude from noisy barometer inputs (raw baro) that is comparable to the reference lighthouse altitude measurement.